Math	Pseudocode
$y = mx + b$	`def line(x,m,b):` `  return m*x + b`

We will also include plenty of inline code (short snippets) and code listings (longer code examples) as well as the code for complete projects. All of the code in the book is provided in Jupyter Notebooks categorized by chapter on the book's GitHub repository (http://mng.bz/JzKp). If you're actively following the text and building the projects in this book, we strongly recommend following the code in this associated GitHub repository rather than copying the code in the text—we will keep the GitHub code updated and bug-free, whereas the code in the book may get a bit out of date as the Python libraries we use are updated. The GitHub code is also more complete (e.g., showing you how to generate the visualizations that we include), whereas the code in the text has been kept as minimal as possible to focus on the underlying concepts.

Since reinforcement learning involves a lot of interconnecting concepts that can become confusing when just using words, we will include a lot of diagrams and figures of varying sorts. The most important kind of figure we'll use is the *string diagram*. It's perhaps an odd name, but it's a really simple idea and is adapted from category theory, a branch of math we mentioned in the first chapter where they tend to use a lot of diagrams to supplement or replace traditional symbolic notation.

You already saw the string diagram in figure 2.1 when we introduced the general framework for reinforcement learning in chapter 1. The idea is that the boxes contain nouns or noun phrases, whereas the arrows are labeled with verbs or verb phrases. It's slightly different from typical flow diagrams, but this makes it easy to translate the string diagram into English prose and vice versa. It's also very clear what the arrows are *doing* functionally. This particular kind of string diagram is also called an *ontological log*, or *olog* ("oh-log"). You can look them up if you're curious about learning more.